

6October2025

October 6, 2025

0.1 Bonding sites

The simulation is run for 10000+100000 timesteps. A total of 300 molecules are used, and each has 4 bonding sites.

0.2 Potential function for patch-bead combination

The potential function, when the patch-bead pairs approach each other patch-first, are

$$V(r) = \text{LJ}(r) + \text{Gaussian}(r)$$

where

$$\text{LJ}(r) = \begin{cases} 4\epsilon \left[\left(\frac{\sigma}{r+x_0} \right)^{12} - \left(\frac{\sigma}{r+x_0} \right)^6 \right] & r < r_c + x_0 \\ 0 & 0 \leq r_c + x_0 \end{cases}$$

and the cutoff distance has been calculated as the point where the potential is 0. The Gaussian is defined as

$$\text{Gaussian}(r) = A \exp \left(-\frac{r^2}{2\sigma_G^2} \right)$$

and the prefactor A is considered an energy scale for the depth of the potential well. Note that, in the region where the Gaussian is effective, the WCA repulsion is also effective. Changing A also moves the position of the minimum, although for most reasonable values this remains below $r = 1.3$, which is the size of our beads.

There were some issues with the proper definition of the variance or standard deviation of the Gaussian. Per our results previously, the most reasonable value is $\sigma_G^2 = 0.19$.

The two parameters varied are A and x_0 . The latter is the distance between the bead and the patch. This is assumed to be constant for the purposes of the analysis, and starts with the value $x_0 = 0.45$

1 Varying energy (A)

Varying the factor in front of the Gaussian varies depth of the potential well. Also varies the shape of the potential function slightly, which is reflected in the bond lengths below.

1.1 Bond length

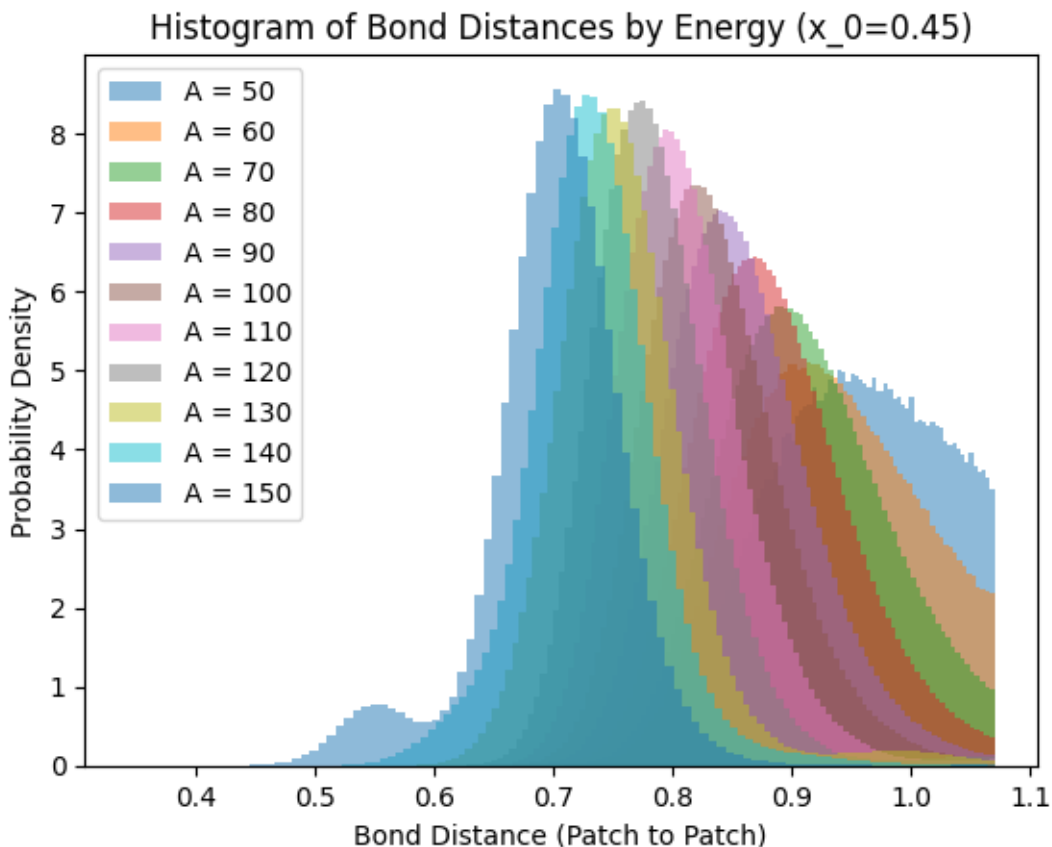
```
[1]: # histogram of bond distances

energies = [50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150]

import matplotlib.pyplot as plt
import pandas as pd

for energy in energies:
    file = f'data/amine_bonds_{energy}.0.csv'
    data = pd.read_csv(file, skiprows=1, delimiter=' ', names=['timestep',
↪ 'id1', 'id2', 'x1', 'y1', 'z1', 'x2', 'y2', 'z2', 'distance'])
    plt.hist(data[data['timestep']>5000]['distance'], bins=100, density=True,
↪ label=f'A = {energy}', alpha=0.5)

plt.xlabel('Bond Distance (Patch to Patch)')
plt.ylabel('Probability Density')
plt.title('Histogram of Bond Distances by Energy (x_0=0.45)')
plt.legend()
plt.xlabel('Bond Distance (Patch to Patch)')
plt.ylabel('Probability Density')
plt.show()
```



As expected, the deeper the potential well the smaller the bond length on average – increasing the potential also moves the well closer to zero. This does definitely serve to manipulate the bond distance between the two patches.

Let's also see the difference of where multiple bonds are in bond length, triple is expected to be higher as there is more steric hindrance for the closer bond lengths.

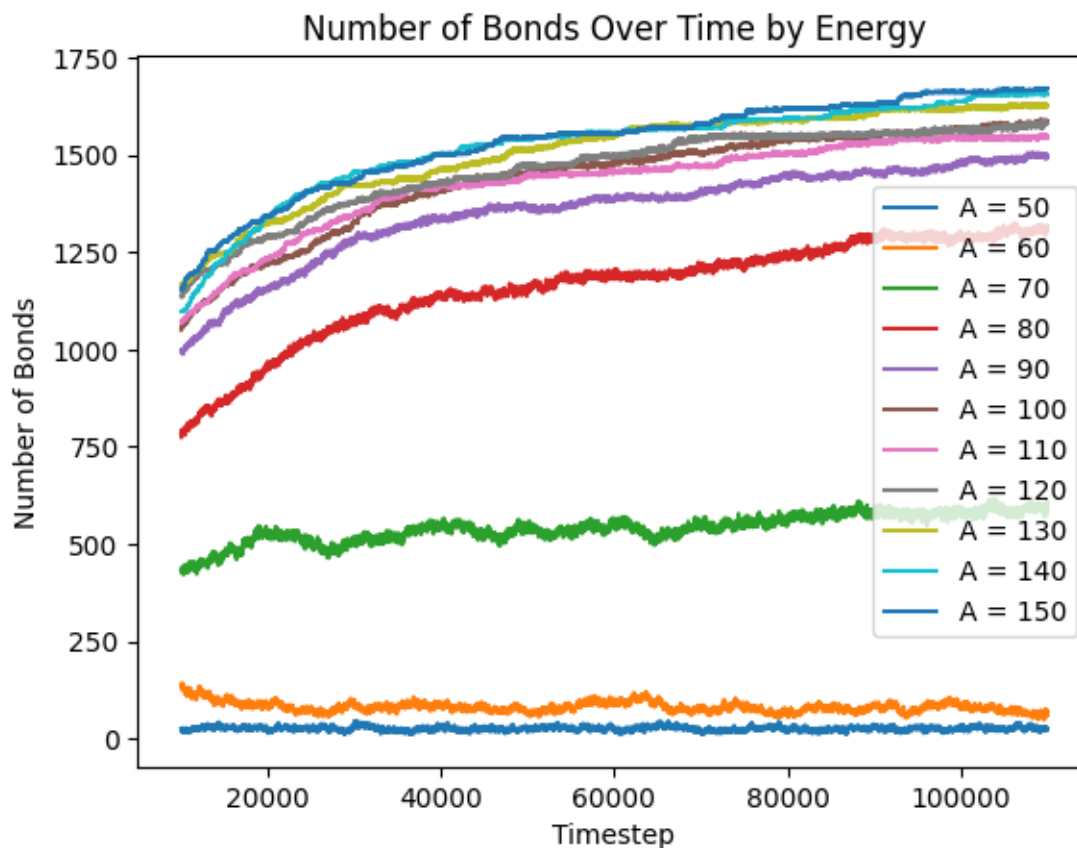
```
[2]: # TODO: add in different bond order vs length analysis
```

1.2 Number of bonds over time

The number of bonds may change over time, which would imply that the system has not reached bond saturation/equilibrium. This would affect the bond distribution/behaviour. The system should be at equilibrium for us to judge it.

```
[3]: # plot number of bonds over time for each energy
for energy in energies:
    file = f'data/amine_bonds_{energy}.0.csv'
    data = pd.read_csv(file, skiprows=1, delimiter=' ', names=['timestep', 'id1', 'id2', 'x1', 'y1', 'z1', 'x2', 'y2', 'z2', 'distance'])
    bond_counts = data['timestep'].value_counts().sort_index()
```

```
plt.plot(bond_counts.index, bond_counts.values, label=f'A = {energy}')
plt.xlabel('Timestep')
plt.ylabel('Number of Bonds')
plt.title('Number of Bonds Over Time by Energy')
plt.legend()
plt.show()
```



It looks like they are for the most part at equilibrium. Some of the higher energy do seem to still be equilibrating at the beginning but seem to taper off by the 1/4 of the timesteps.

We do see that the lower values of A have far fewer counts of bonds.

1.3 Multiple patch bonds

Multiple bonds are the issue with this model. There is an acceptance angle where two or more patches may approach a single patch and still be favourable. These are meant to be prevented by the steric hindrance of having two patches in a bond, where the two attached beads are preventing other patches from approaching. Therefore, we can reason that this effect is stronger if the patch-patch bond length is shorter – the repulsions are stronger close to the bonded patch pair, and the incoming patch needs to get closer in order to be bonded.

It will be useful to calculate the probability of forming 3-patch bonds if there are currently 2-patch bonds. It may also be possible that 3-patch bonds occur when they are made at the same time.

```
[4]: # TODO: calculate probability of forming 3-patch bonds if there are currently
      ↪ 2-patch bonds

import matplotlib.pyplot as plt

avg_neighbours = []
for energy in energies:
    file = f'data/amine_multiplets_{energy}.0.csv'
    data = pd.read_csv(file)
    data['num_neighbours'] = (data.count(axis=1) - 2) / 4 # subtract timestep
    ↪ and center columns
    neighbour_counts = data['num_neighbours'].value_counts().sort_index()
    norm_counts = neighbour_counts / neighbour_counts.sum()
    # plt.hist(data['num_neighbours'].dropna(), bins=range(2, 12), alpha=0.5,
    ↪ label=f'Energy {energy}', align='left')
    plt.plot(range(2, len(norm_counts) + 2), norm_counts, label=f'A = {energy}')
    print(f"A = {energy} - Average number of neighbours in multiplets:",
    ↪ data['num_neighbours'].mean())
    avg_neighbours.append(data['num_neighbours'].mean())
    print(f"A = {energy} - Distribution of neighbours in multiplets:")
    print(data['num_neighbours'].value_counts().sort_index())
plt.xlabel('Number of Neighbours in Bond')
plt.ylabel('Frequency')
plt.title('Distribution of # Neighbours in Bond')
plt.legend()
plt.show()
```

```
A = 50 - Average number of neighbours in multiplets: 2.018077377935462
A = 50 - Distribution of neighbours in multiplets:
num_neighbours
2.0    5812
3.0     107
Name: count, dtype: int64
A = 60 - Average number of neighbours in multiplets: 2.1765031680623865
A = 60 - Distribution of neighbours in multiplets:
num_neighbours
2.0    74345
3.0    15928
4.0         3
Name: count, dtype: int64
A = 70 - Average number of neighbours in multiplets: 2.522731981307981
A = 70 - Distribution of neighbours in multiplets:
num_neighbours
2.0    609282
3.0    642488
```

```

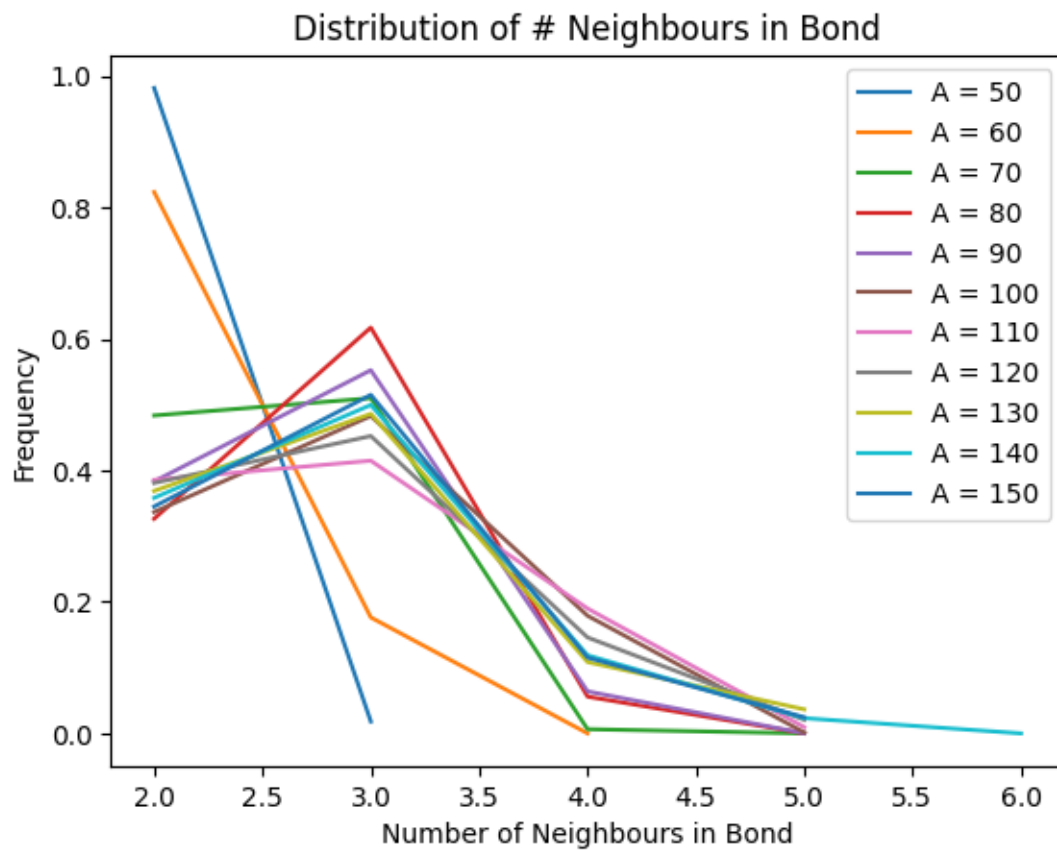
4.0      8013
5.0        6
Name: count, dtype: int64
A = 80 - Average number of neighbours in multiplets: 2.729505445709521
A = 80 - Distribution of neighbours in multiplets:
num_neighbours
2.0      819317
3.0     1548934
4.0     140316
5.0        197
Name: count, dtype: int64
A = 90 - Average number of neighbours in multiplets: 2.6807812654171816
A = 90 - Distribution of neighbours in multiplets:
num_neighbours
2.0     1146162
3.0     1652142
4.0     191149
5.0         312
Name: count, dtype: int64
A = 100 - Average number of neighbours in multiplets: 2.8450024828177725
A = 100 - Distribution of neighbours in multiplets:
num_neighbours
2.0     1193779
3.0     1711218
4.0      634314
5.0         5049
Name: count, dtype: int64
A = 110 - Average number of neighbours in multiplets: 2.8232305743207835
A = 110 - Distribution of neighbours in multiplets:
num_neighbours
2.0     1379945
3.0     1484463
4.0      679683
5.0         33892
Name: count, dtype: int64
A = 120 - Average number of neighbours in multiplets: 2.806394055177999
A = 120 - Distribution of neighbours in multiplets:
num_neighbours
2.0     1337832
3.0     1588643
4.0      512640
5.0         72647
Name: count, dtype: int64
A = 130 - Average number of neighbours in multiplets: 2.8127847307768
A = 130 - Distribution of neighbours in multiplets:
num_neighbours
2.0     1214492
3.0     1600367

```

```

4.0      357380
5.0      120345
Name: count, dtype: int64
A = 140 - Average number of neighbours in multiplets: 2.806905636242731
A = 140 - Distribution of neighbours in multiplets:
num_neighbours
2.0      1268907
3.0      1769540
4.0       420953
5.0        82046
6.0           9
Name: count, dtype: int64
A = 150 - Average number of neighbours in multiplets: 2.8191501527409106
A = 150 - Distribution of neighbours in multiplets:
num_neighbours
2.0      1198262
3.0      1787984
4.0       400208
5.0        85111
Name: count, dtype: int64

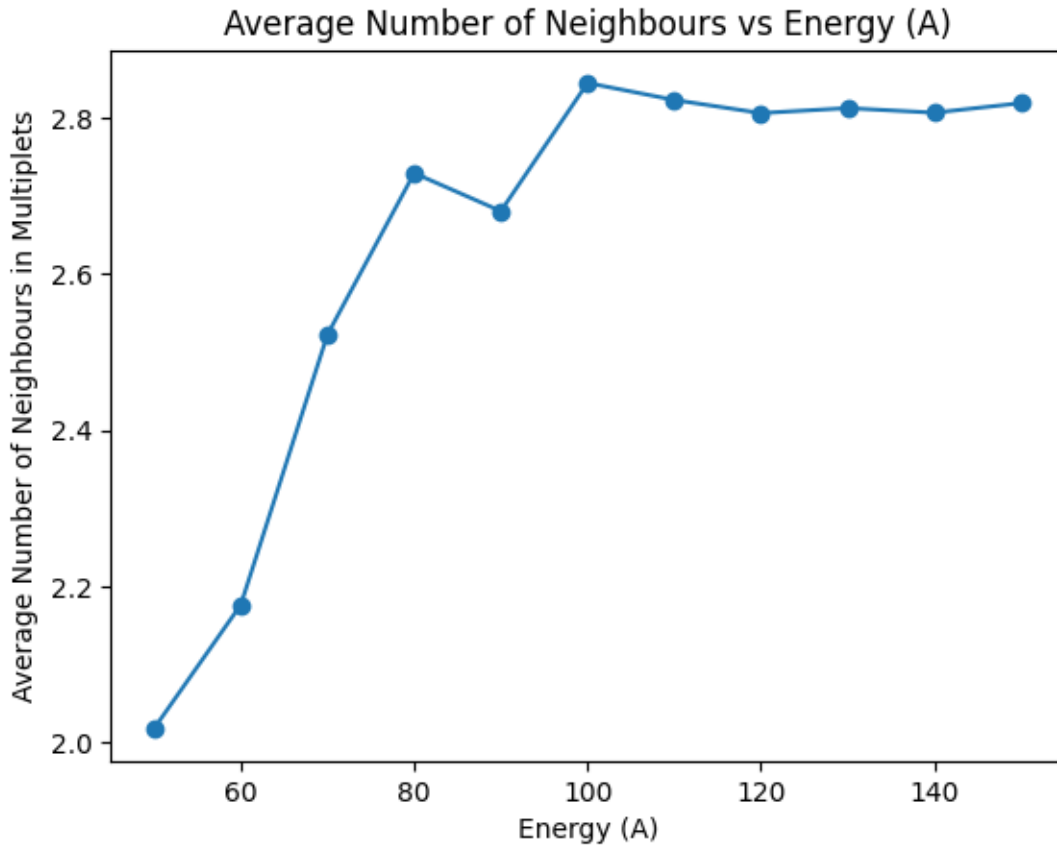
```



```
[5]: # graph average number of bonds

import matplotlib.pyplot as plt

plt.plot(energies, avg_neighbours, marker='o')
plt.xlabel('Energy (A)')
plt.ylabel('Average Number of Neighbours in Multiplets')
plt.title('Average Number of Neighbours vs Energy (A)')
plt.show()
```



Each bonded pair is grouped with the other bonded pairs such that an adjacency list is formed with each group accounting for all interconnected bonds.

The distribution graph was my initial attempt to tell at a glance which A value works best. The most desirable would not appear on the graph at all – if all the bonds are 2-patch, they wouldn't show up in the line plots. However, all of them have at least some 3-patch bonds.

The best option is $A = 50$, where there are only 2-, 3-patch bonds and there are far more 2-patch bonds than 3-. The proportion of 2-patch bonds is $\frac{5812}{(5812+107)} \approx 0.981$ which is the vast majority of the bonds.

Similarly, $A = 60$ is close with almost negligible amounts of 4-patch bonds (proportion 0.00003),

with 2-patch bonds having a proportion of 0.824. Although this is a majority, there is a non-negligible amount of 3-patch bonds already. Greater A values are worse.

This is a bit easier to see in the average number of neighbours plot. Here, the average starts at almost 2 and increases until a plateau past $A = 100$. Generally, each subsequent greater A value is worse with less 2-patch bonds in proportion to all the bonded groups.

2 Varying patch-bead length (x_0)

Although not directly changing the Gaussian strength, moving the patch closer and further from the bead still affects the bond length and strength.

2.1 Bond length

Once again, we can see the different bond lengths as a way to visualize the potential (is this true?).

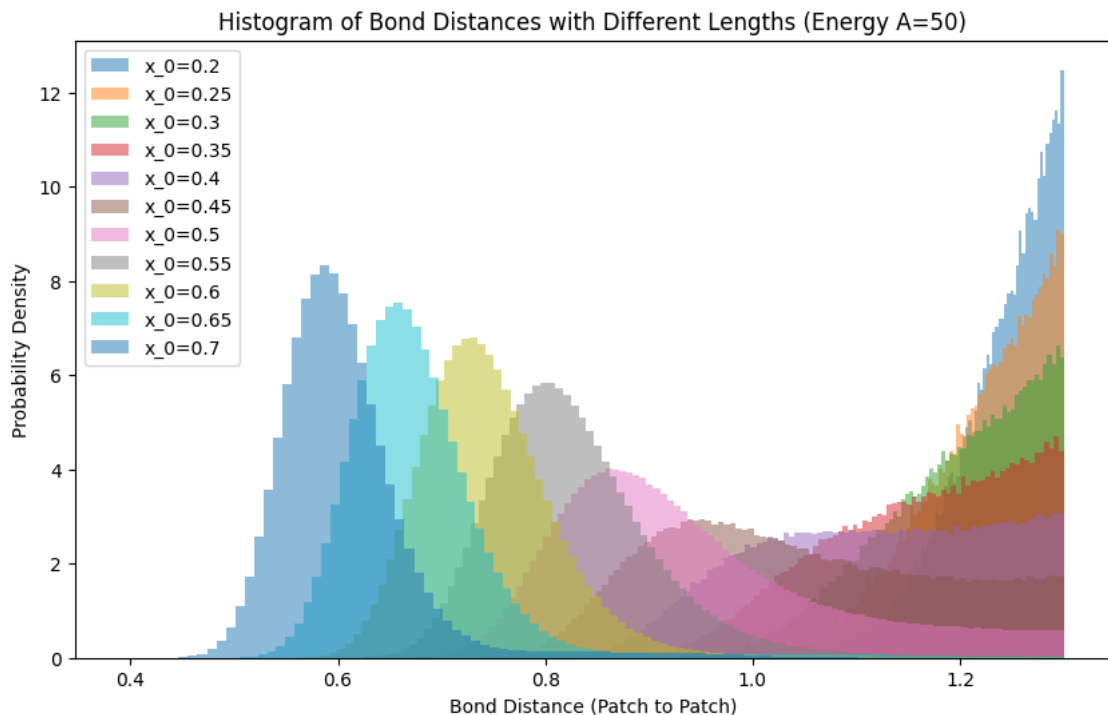
```
[6]: # histogram of bond distances

lengths = [0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.7]

import matplotlib.pyplot as plt
import pandas as pd

figure = plt.figure(figsize=(10, 6))
for length in lengths:
    file = f'data/amine_bonds_length_{length}.csv'
    data = pd.read_csv(file, skiprows=1, delimiter=' ', names=['timestep', '
    ↪id1', 'id2', 'x1', 'y1', 'z1', 'x2', 'y2', 'z2', 'distance'])
    plt.hist(data[data['timestep']>5000]['distance'], bins=100, density=True,
    ↪label=f'x_0={length}', alpha=0.5)

plt.xlabel('Bond Distance (Patch to Patch)')
plt.ylabel('Probability Density')
plt.title('Histogram of Bond Distances with Different Lengths (Energy A=50)')
plt.legend()
plt.xlabel('Bond Distance (Patch to Patch)')
plt.ylabel('Probability Density')
plt.show()
```

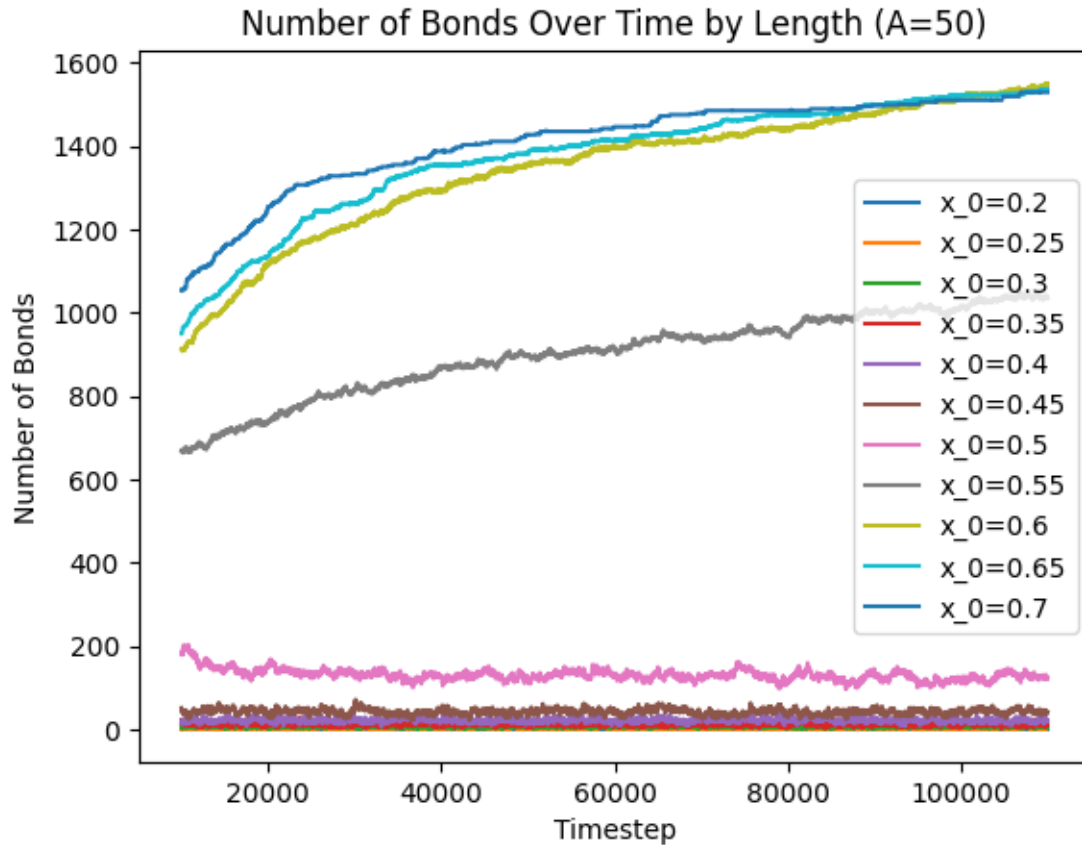


(Note that the above is different from [Section 1.1](#) as the Bond Distance cutoff is at 1.3, instead of 1.1)

Similar to varying energy, we get a spread of values that get more tightly peaked and closer to 0 as length $x = 0$ increases. As length $x = 0$ increases, the Gaussian moves towards the other patch. The potential well also becomes deeper as there is less repulsion experienced at that bond length. At the very low lengths (x_0 0.3, close to bead size) the variance gets quite spread and it is, effectively, just the WCA repulsion that takes over. Note that the values closer to 1.3 begin to peak again because the graph is probability densities and the range of values is getting smaller.

2.2 Number of bonds over time

```
[7]: # plot number of bonds over time for each length
for length in lengths:
    file = f'data/amine_bonds_length_{length}.csv'
    data = pd.read_csv(file, delimiter=' ')
    bond_counts = data['timestep'].value_counts().sort_index()
    plt.plot(bond_counts.index, bond_counts.values, label=f'x_0={length}')
plt.xlabel('Timestep')
plt.ylabel('Number of Bonds')
plt.title('Number of Bonds Over Time by Length (A=50)')
plt.legend()
plt.show()
```



Only values of $x_0 > 0.5$ have any kind of change in number, the other values are all fairly consistent in the number of bonds. However, the other values do have a fairly low number of bonds in general.

2.3 Multiple patch bonds

```
[8]: import matplotlib.pyplot as plt

avg_neighbours = []

for length in lengths:
    file = f'data/amine_multiplets_length_{length}.csv'
    data = pd.read_csv(file)
    data['num_neighbours'] = (data.count(axis=1) - 2) / 4 # subtract timestep
    # and center columns
    neighbour_counts = data['num_neighbours'].value_counts().sort_index()
    norm_counts = neighbour_counts / neighbour_counts.sum()
    plt.plot(range(2, len(norm_counts) + 2), norm_counts, label=f'Length_{length}')
    avg_neighbours.append(data['num_neighbours'].mean())
```

```

    print(f"Length {length} - Average number of neighbours in multiplets:",
    ↪data['num_neighbours'].mean())
    print(f"Length {length} - Distribution of neighbours in multiplets:")
    print(data['num_neighbours'].value_counts().sort_index())
plt.xlabel('Number of Neighbours in Bond')
plt.ylabel('Frequency')
plt.title('Distribution of Neighbours in Bond')
plt.legend()
plt.show()

```

```

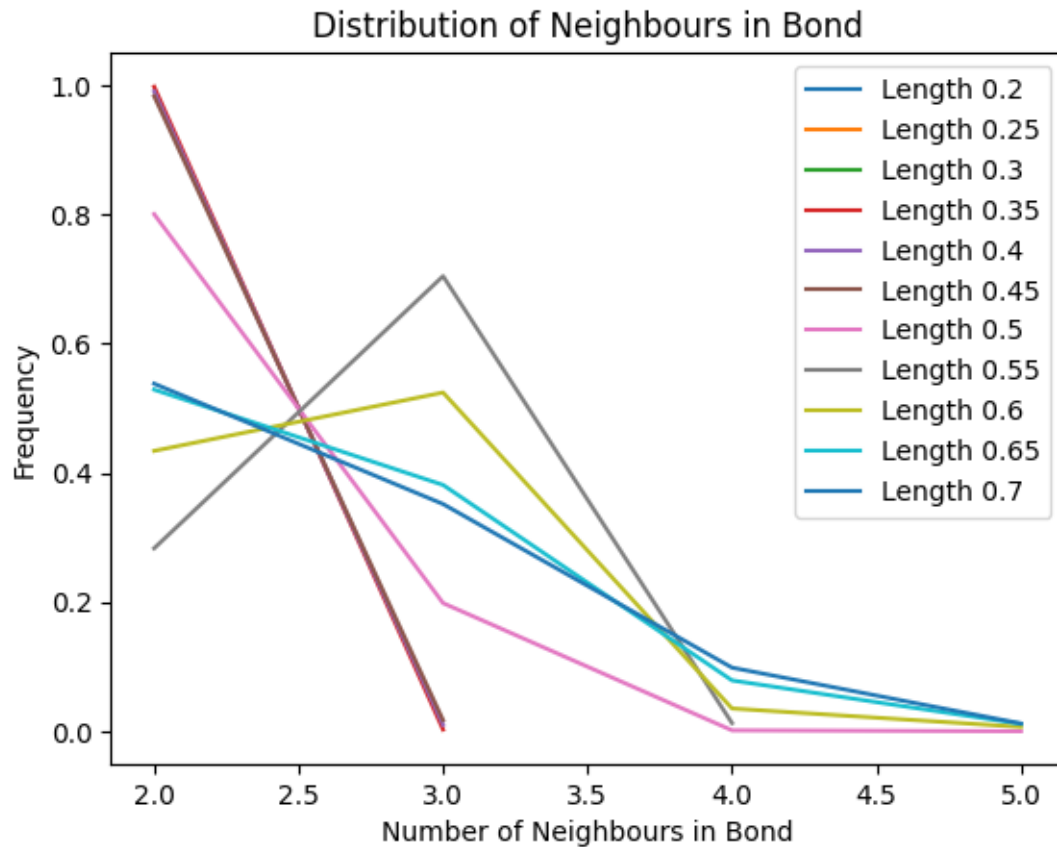
Length 0.2 - Average number of neighbours in multiplets: 2.0
Length 0.2 - Distribution of neighbours in multiplets:
num_neighbours
2.0    103
Name: count, dtype: int64
Length 0.25 - Average number of neighbours in multiplets: 2.0
Length 0.25 - Distribution of neighbours in multiplets:
num_neighbours
2.0    219
Name: count, dtype: int64
Length 0.3 - Average number of neighbours in multiplets: 2.0
Length 0.3 - Distribution of neighbours in multiplets:
num_neighbours
2.0    583
Name: count, dtype: int64
Length 0.35 - Average number of neighbours in multiplets: 2.0025906735751295
Length 0.35 - Distribution of neighbours in multiplets:
num_neighbours
2.0    1155
3.0      3
Name: count, dtype: int64
Length 0.4 - Average number of neighbours in multiplets: 2.009806386723661
Length 0.4 - Distribution of neighbours in multiplets:
num_neighbours
2.0    3938
3.0     39
Name: count, dtype: int64
Length 0.45 - Average number of neighbours in multiplets: 2.017026988837078
Length 0.45 - Distribution of neighbours in multiplets:
num_neighbours
2.0    13913
3.0     241
Name: count, dtype: int64
Length 0.5 - Average number of neighbours in multiplets: 2.2012243488050767
Length 0.5 - Distribution of neighbours in multiplets:
num_neighbours
2.0    125100
3.0     31003

```

```

4.0      221
5.0       4
Name: count, dtype: int64
Length 0.55 - Average number of neighbours in multiplets: 2.7294254035614505
Length 0.55 - Distribution of neighbours in multiplets:
num_neighbours
2.0      467685
3.0     1163435
4.0      20735
Name: count, dtype: int64
Length 0.6 - Average number of neighbours in multiplets: 2.6148000525442217
Length 0.6 - Distribution of neighbours in multiplets:
num_neighbours
2.0     1195634
3.0     1444546
4.0       97078
5.0       18516
Name: count, dtype: int64
Length 0.65 - Average number of neighbours in multiplets: 2.57277024724916
Length 0.65 - Distribution of neighbours in multiplets:
num_neighbours
2.0     1409060
3.0     1015864
4.0       209669
5.0        30411
Name: count, dtype: int64
Length 0.7 - Average number of neighbours in multiplets: 2.5845581029303
Length 0.7 - Distribution of neighbours in multiplets:
num_neighbours
2.0     1440832
3.0       941833
4.0       263674
5.0        32192
Name: count, dtype: int64

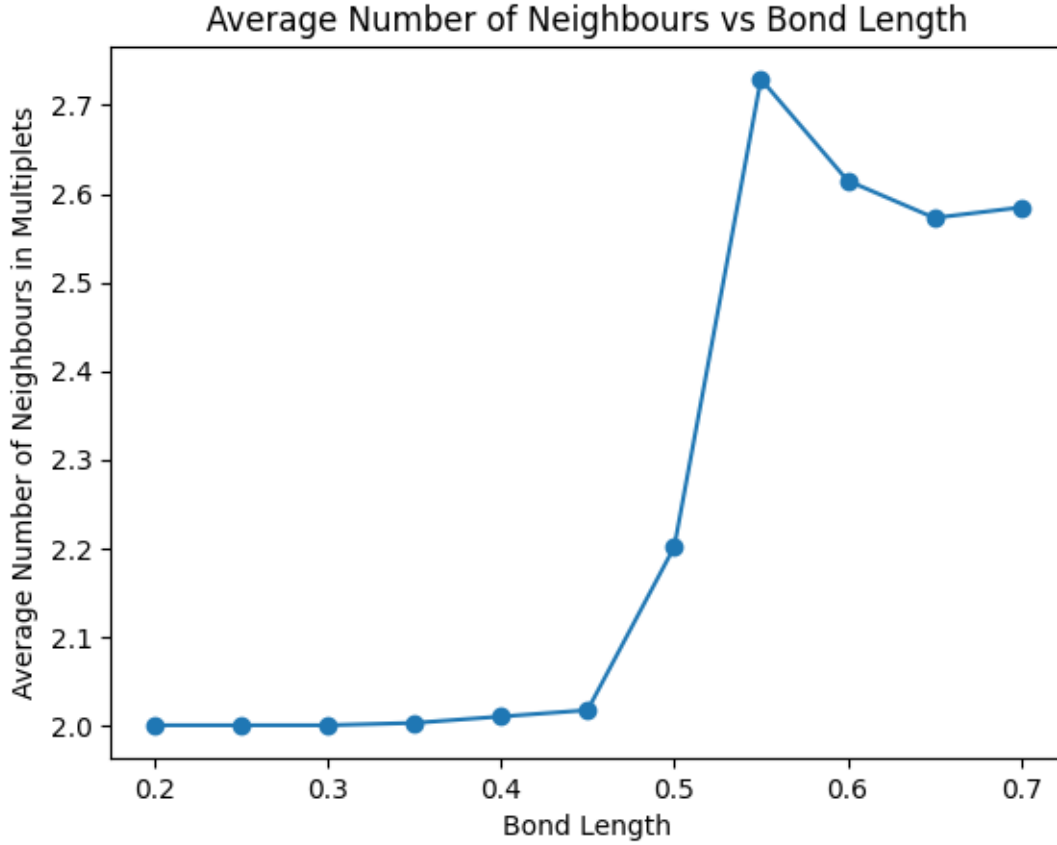
```



```
[9]: # graph average number of bonds

import matplotlib.pyplot as plt

plt.plot(lengths, avg_neighbours, marker='o')
plt.xlabel('Bond Length')
plt.ylabel('Average Number of Neighbours in Multiplets')
plt.title('Average Number of Neighbours vs Bond Length')
plt.show()
```



Similar to varying energy, the values where the potential is tightly peaked closer to 0, there are less patches in each bond. This is due to the lower acceptance angle when moving the Gaussian closer to the center of the WCA repulsion.

The values of average number of neighbours monotonically increases for the first couple values, ranging from $x_0 = 0.2$ with only 2-patch bonds, to $x_0 = 0.45$ with a proportion of about 0.983 for 2-patch bonds (note that this is the same case as $\$A=50$, $x_0 = 0.45$ in Section 1). After that, the next length already approaches unfeasible proportions ($x_0 = 0.5$ with proportion of 2-patch bonds about 0.800). Another plateau occurs at high lengths.